ResearchGate

See discussions, stats, and author profiles for this publication at: https://www.researchgate.net/publication/271425337

C++ or Python? Which One to Begin with: A Learner's Perspective

Conference Paper · April 2014

DOI: 10.1109/LaTiCE.2014.20

CITATIONS
31

READS
3,703

4 authors, including:



Muhammad Ateeq
The Islamia University of Bahawalpur
13 PUBLICATIONS **124** CITATIONS
SEE PROFILE



Adnan Umer
Information Technology University of the Punjab
2 PUBLICATIONS **35** CITATIONS
SEE PROFILE



Muzammil Ur Rehman
The Islamia University of Bahawalpur
12 PUBLICATIONS **71** CITATIONS
SEE PROFILE

All content following this page was uploaded by Muhammad Ateeq on 01 April 2015.

The user has requested enhancement of the downloaded file.

C++ or Python?

Which One to Begin With: A Learners Perspective

Muhammad Ateeq, Hina Habib, Adnan Umer, Muzammil Ul Rehman

Department of Computer Science & IT
The Islamia University of Bahawalpur
Pakistan
rmateeq@yahoo.com, hinahabib02@gmail.com, adnanumer95@gmail.com, murq786@yahoo.com

***Abstract*---Programming is pivotal to the field of Computing. C++ and Java have long been choice for an introductory programming course. Off late the trend has been to adopt Python as an introductory programming language for various reasons. In this paper we present results of a study on comparison of C++ and Python as an introductory programming language. Results indicate that due to its simpler, pseudocodish syntax, easy to learn environment and higher abstraction more student choose Python as an introductory programming language compared to C++.**

Keywords---C++; Python; Introductory Programming; Learner

I - INTRODUCTION

Choosing a right medium for an introductory programming course has been debated widely. It is important that the language chosen to teach programming to novices' leaves a positive impact on learner and encourages them to continue to learn, and motivates them to adopt programming as their career if they wish so. The first language learnt definitely has an influence on the programming style and coding technique that a learner develops and continues throughout the journey [3][5][8][9]. We have found abundant studies on the choice of introductory programming language discussing the basic feature set of several languages, problems faced by the students while learning a new language, comparisons between the candidate languages and the methods of how to introduce/teach an introductory language [2][5][12][10]. In an introductory course, major aim is to teach novices the art of problem solving and presenting solutions in a simple and precise way [1]. So a common recommendation would be to teach a language that is easy to understand, is stable, has excellent support, is versatile and whose application scope is not limited to a single area. Some of the aspects discussed in literature are regularity, level of difficulty, turnaround time, error detection and debugging support, verbosity, ease of reuse, data types and student satisfaction [2][5]. Today, C, C++ and Java are amongst the most popular languages widely taught at universities. A common feedback about these languages is that these are too complicated and verbose with notational overhead [2]. More over the students find it hard to deal with the abstract concepts [2]. The problem does not lie merely in the languages themselves but the context in which they are going to be used is also significant [3].

Teaching of programming in the local context is started at school level, and is available at all levels of education afterwards, but optionally. Although a few of the students have already learnt some programming language before getting enrolled in CS degree program at university level, and it is believed that prior programming has a positive impact on CS studies [20], but for most of the students it is the very first time to encounter programming and problem solving. Moreover it is common and rational to teach programming at university level with the assumption that no one has studied programming prior to this.

A. Difficulties

Identifying the difficulties faced by novices in learning programming has long been studied. Lister et al. [17] in their results from a multi-national study figured out that students commonly lack in tracing the execution of even short pieces of code having studied their first course on programming. Difficulty in learning abstract concepts has been reported widely too. A comprehensive review has been presented in [18].

Code Listing 1 -- Comparison of Code for simple addition and output in Python and C++

Python

```
x = 2
y = 3
z = x + y
print(z)
```

C++

```
#include<iostream>
int main() {
        int x, y, z;
        x = 2;
        y = 3;
        z = x + y;
        cout<<z<<endl;
}
```

978-1-4799-3592-5/14 $31.00 © 2014 IEEE
DOI 10.1109/LaTiCE.2014.20

IEEE computer society

B. *Programing Language and Learning*

Here, we are particularly interested in discussing issues related to languages in learning programming. According to Milbrandt [19], while selecting a programming language for educational purpose, basic features that one should consider, include ease of learning, structured design, wide applicability, power, simple syntax, easy I/O handling, meaningful keywords and immediate feedback, among others.

In recent times C++ and Java are the most popular languages as a choice to teach novices. Python is also catching up fast. As mentioned earlier, factors that push a language back include verbosity and notational overhead. It is believed that these factors make it difficult for a novice to focus problem solving since it takes considerable time to understand afore mentioned concepts.

II - MOTIVATION

Historically, C++ is the language taught to novices at our department for a long time. We have recently introduced Python in the first semester of CS undergraduate program. This course on Python was followed by C++ in second semester.

Python is a High-level interpreted language, designed by Guido van Rossum [25] back in 1991. In recent years Python has been receiving an ever increasing attention, both in academic and professional community [26]. It has also influenced design of some other popular languages like Ruby and Go [27]. We present some of the key aspects of Python that make it a suitable language for teaching programming to novices, as described in [2]:

- Python offers a clean and more intuitive syntax compared to other contemporary competitive languages for the purpose, like C++ and Java. Code Listing 1 lists sample code for basic arithmetic and output in Python and C++.
- Python is dynamically typed, which reduces the notational overhead and adds to the ease for a novice to focus problem solving rather than language.
- Python offers better expressiveness by introducing powerful data types likes lists and dictionaries that can be taught simultaneously with other basic data types with ease. Python also has a very high statement ratio with C as reference.
- Interpreted nature of Python yields immediate feedback. Not just programming becomes fast but it helps in coping with errors in a better and simpler way.
- Without any use of extra brackets and semicolons Python enforces structural design by enforcing indented and structured way of writing programs. This feature gives Python more resemblance with pseudocode.
- Although irrelevant for novice programmers, but Python offers extendibility to language features due to its 2-tier language design.
- Its default programming environment (IDLE) is open source and free. It can also run on most of the platforms.

Story of computing lies in trade-offs. We also list some of the limitations associated with Python when compared to other languages:

- Where dynamic typing is an advantage, it can also be an error prone and create ambiguities for the programmer.
- Due to 2-tier hierarchy Python can suffer performance loss. But it is irrelevant for small programs that beginners develop.

Currently Python is being used for teaching various CS related courses including introductory courses by the most popular online communities like coursera [22], udacity [23] and edx [24], where most of the leading US universities and others offer their free online courses. In addition, many institutes claim to have earned their success with Python as a first programming language [14][15].

III - STUDY AND METHODOLOGY

Introductory programming course has been taught with C++ exclusively at our department. It has always been observed that students while going through their first course on programming have to face certain difficulties in grasping even the basic concepts due to various factors including lack of prior programming background. We believe, when they encounter a language that has toughness of its own, it leads them to many difficulties and setbacks and potentially put them off the programming path. Many problems have been pointed out in literature in this regard [6][7][11].We considered it worth experimenting to change the tradition of choosing C++ as first language and decided to introduce Python. C++ was introduces later in second semester.

At the end of 2nd semester, with experience of two languages at hand, the students were capable enough to give us some value able data for initial analysis on the topic. We conducted a survey to capture feedback regarding suitability of a programming language for teaching novices with a focus on comparison between the two taught languages based on the factors laid out in research [2][5]. We hope to present the perspective of the learners about impact of a language on their programming progress.

	Python		C++	
Construct	**Mean**	**Std Dev**	**Mean**	**Std Dev**
Algorithms	2.64444	0.88306	2.5556	1.0347
Variables	2	0.87905	2.2222	0.9975
List/Arrays	2.75556	1.19003	2.6889	1.0406
Boolean Expressions	2.47619	1.08736	2.5714	1.1293
For Loop	2.04444	0.76739	2.7273	1.0424
While Loop	2.15556	0.9524	2.6818	1.0292
If Statement	1.84091	0.68005	2.1429	0.9258
Functions	2.95556	1.0435	2.7619	0.983
Files	3.75	0.98058	---	---

Table 1: The perceived difficulty level of language constructs (1 = very easy, 5 = very difficult)

As against a conventional setting a study is conducted on two parallel groups, where one group studies C++ and the other Python, we instead decided to take the idea horizontally. Primary reason for this was that this was the first ever experience of teaching Python in our department. This study was conducted at the end of the course on C++. Both the courses were taught with a 90 minutes lecture twice a week for 14 weeks. For the purpose of our research we conducted a survey among 45 students. The survey consisted on two parts. First part was quantitative where our focus was to seek feedback on difficulty of various syntactic and overall features of both languages. The second part consisted of open ended questions. The focus in this part was to get an insight on satisfaction level as well as perception about first programming language among students and impact of studying one language on the other. The survey was distributed among students manually and selected students were interviewed.

IV - RESULTS

In this section we discuss results from our survey regarding choice of first programming language. It must be considered that students studied Python with no assumed background in programming, whereas while studying C++, students already had exposure of Python. So while studying Python, students were considered new to programming, whereas while studying C++ they already had exposure to programming. This is an important consideration while comparing statistics presented below.

A. Language Features Comparison

Table 1 presents comparison of difficulty of various language constructs provided by C++ and Python. Results suggest that students found overall algorithms design and functions a little more difficult in python when compared with C++, but it can be justified since most of the students were new to programming when they studied Python. It was difficult to think about problem solving and design an overall algorithm without any prior knowledge. It should be noted that user defined modules were introduced at very beginning of the course in Python. These factors can have an effect on students learning capacity and opinion. Another reason could be that during the first course, students were making quite simple and small programs so they failed to understand why functions were needed at all. Similar is the case with lists. Lists were also introduced early in the course in case of Python whereas in case of C++ lists are introduced almost after half of the course is covered already. It is also important to note that some of the students (12 out of 45, 27%) had prior programming exposure and the sole language taught optionally in the prior schooling is C++.

From table 1 it is also clear that generally stdev is higher for more constructs of C++. That also is an indicator of effect of prior experience of some of the students. However in case of loops and conditional statements students seemed to enjoy the ease and intuitive style provided by Python. As has been identified in previous research [16] that students find it useless to write comments and documentation for small programs, likewise they probably think of writing documentation as time consuming and not so useful. It looked as if students didn't appreciate this activity may be because they were young and not used to writing thorough documentation and had not yet developed the necessary skill. In case of C++ students had not studied file handling, so there couldn't be any comparison made.

	Python		C++	
Feature / Aspect	**Mean**	**Std Dev**	**Mean**	**Std Dev**
Ease to Program	2.47727	0.731	2.6818	0.9092
Expressiveness	2.76923	0.70567	2.8	0.791
Simplicity	2.44444	1.05649	2.6279	1.1552
Richness of Modules	2.4419	0.90219	2.7404	0.9386
Flexibility	2.57778	0.72265	2.8605	0.8614
Satisfaction	2.57778	0.86573	2.9556	1.0435
Debugging Facility	2.42222	1.01105	2.6744	0.9933
Compactness	2.51111	0.89499	2.8667	1.0135

Table 2: Comparison of various aspects for programmer (1 = very high, 5 = very low)

B. Other General Aspect

Table 2 presents view of students about some of the important aspects one can consider while learning a programming language. Although C++ was taught with Python already taught, still students felt it was a touch easier to program in Python compared to C++. This is consistent with what we expect from Python. In all aspects Python is considered a little better than C++. We see a large value of deviation in case of "Simplicity". This is probably because we witnessed a difference of opinion when defining simplicity. For some simplicity was through 'abstraction' while others see 'clarity' making more sense to them and hence making the language look simpler. For example when students were asked about dynamic typing of variables in Python, some considered it easy, while others though that hiding this information makes the concept of variables ambiguous and difficult to comprehend. Students showed more satisfaction with Python.

Another feature of Python that they appreciated was that it can be used as an interactive shell. When we talked about flexibility, most students seemed to have been pushed back by flexibility provided by C++ of being able to do the same thing in various ways. For them it created a bit of confusion and vagueness. Flexibility is appreciable and productive but only after having gained due experience of programming. There is a general trend observed that beginners make small changes and debug programs by making small changes at a time, then re-compile and see the execution. Since Python is a scripting language, thus one can save time in writing and executing short programs over and over again. So again students were in favor of Python regarding debugging.

Again we witness a high value of stdev for more cases of C++, which is potentially an indicator of variations in opinion as some of the students had prior programming experience in C++, and is consistent with the values exhibited in Table 1.

V - DISCUSSION

We now present a detailed analysis of our findings including the results of the selected interviews that we conducted to gain further insight of the students' opinions.

A. Satisfaction

In interview feedback, almost half (8 out of 17) of the students were satisfied with their progress in learning programming. Most of the students attributed their satisfaction/dissatisfaction to lack of practical work/lack of lab sessions (practical work).

Other main factors of satisfaction included background in programming and self-interest. Likely the factors for dissatisfaction included not giving enough time, teaching methodology and lack of practice. Three students mentioned Python as a factor of satisfaction in progress of learning programming and one, C++. Many students felt that, it had little to do with language choice as far as learning programming is concerned. One of the students commented:

> *"... as long as practical work is focused, no matter what the language is, we can grasp the ideas and work with it."*

So it can be concluded that practice brings perfection and hence satisfaction.

B. Students Choice for First Programming Language

The first programming language that one ever learns is probably the hardest one. The learners' job is to learn language and problem solving skills simultaneously. For a

beginner, to choose a language that's more interesting, intuitive and fun can make it more like an adventure than a challenge thus increasing the chances of getting involved in it. 60% students voted Python to be the best choice as a first programming language to be learned. The reasons narrated included ease, simpler syntax, better understanding, programming environment and abstraction. One of the students commented:

> *"... I will surely go for Python because the kind of liberty, ease and friendliness that it provides to a beginner compared to C++, is matchless. It is fun and definitely it encourages me to continue with it."*

Only one out of the rest stated that the language by itself is not an issue. It doesn't matter where you start until you keep going. It is hard to learn languages for their own merits without the existence of some other interesting goal like building something new. Those who supported C++ were mostly those, who had some previous experience with the language. And they all had one common argument that C++ has much less abstraction than Python and hence making it easy to understand the concepts. This shows influence of prior knowledge of C++. It is important to note that clarity of concepts and a closer interaction with machine is ranked higher by many student programmers instead of leaving the things unrevealed under the umbrella of abstraction. One interesting fact about these students is that almost 80% of them after having completed 2nd course on the language of their choice, i.e. C++, they still felt dissatisfied with their performance. That surely implies that not just the choice of programming language but many other factors lead to a better output.

C. *Students Perception of a first programming language*

We asked respondents what ideas and expectations were there in their mind when they began the first course, and if they had witnessed big deviances from their initial perception. For them making a smart choice was important, because it certainly affects your progress, and a wrong choice could have caused loss of motivation. They also had some bigger goals in mind like being able to secure a perfect place in the job market in the future. But for a safer start they all mentioned that the most important aspect they thought was the level of ease that a language could offer. They expected simpler syntax, human readability, appealing environment and intuitiveness. They generally anticipated that they would be able to do something useful or fun right after a few initial lectures. Python for sure served the purpose for most of them. We further inquired what 'ease' of a programming language could mean to a beginner. Diverse answered were received. For some, ease of programming merely means simple syntax while for others flexibility, readability, built-in-functions and richness of modules makes usage of language easy. A beginner can easily be over whelmed by code that is unintuitive or hard to read. Languages with unusual syntax are difficult to learn. If the language turns out to be difficult, it can cause students to feel uncomfortable and hence can be a potential cause of drop out of the course.

D. *Future Use*

The likelihood for usage of either language turned out to be almost equal. C++ is deep rooted and has already established a massive reputation for having offered a successful future for its users. Nonetheless Python too looks very tempting as a multi-paradigm language. Its evolution has not come to an end and has kept enticing programmers in the private sector [3]. C++ has freshly been taught to the students under our study and 52% of them feel more comfortable when it comes to choosing it as the best option for their future projects. It is also notable that after completing their first course with C++, students are already enrolled to study Object Oriented flavor of C++ in next semester. After having experienced programming in the two languages 64% are confidently ready to keep using Python later in their progress. The figure is big enough to establish that Python is not only being appreciated as a medium to introduce novices to programming, but also being valued for its future use in the industry. It is also notable that those who voted for C++, had prior experience with the language, whereas most of the voters of Python were new to programming.

E. *Language Choice and Problem Solving*

The purpose of an introductory programming course is to familiarize the students to the art of problem solving and to teach them how to express solutions in a simple way. The purpose is to make them capable of thinking algorithmically [1], and not making them bother about the intricacies of a language. Research studies have suggested that problem solving and programming should be treated separately so that the learner can focus one thing at one time [21] while our survey results reveal that 75% participants find it more beneficial to take both the aspects side by side. According to them it doesn't sound fruitful to simply work on problem solving without seeing its practical implementation. One of the students commented:

> *"...I think I cannot feel satisfied until I see the results of what I have done. Only after witnessing the results I shall be sure that what I have done is right."*

So in this situation the choice again should be a language that is simple and easy to learn without many complicated concepts in it, like Python.

VI - CONCLUSION

Choice of a programming language for an introductory programming course requires care. From our findings it is clear that choice of a language can impact learning, survival and progress of students in the field of Computing with programming at its center. Primary thing that students expect from an introductory language is ease. Although significance of C++, Java and other languages cannot be overlooked, Python looks a very strong and viable choice when it comes to introductory programming course. Students show more satisfaction while learning Python and feel that most of the language features including loops and conditionals were easier compared to C++.

This survey had a limited scope with some factors not included. We did not include grades of students due to some potential circumstantial inconsistencies and other internal reasons. It would had been worthwhile to conduct a cross study with C++ taught first and Python later, but at this stage it was not possible since Python was introduced for the very first time. In future we want to extend our work in various ways. First, we want to monitor the performance of students through a programming test designed in both languages in order to better trace learning of a student in any particular language. Second, since we have new enrollments in CS undergraduate programs, we want to experiment with introducing one section of students to GUI based game programming in Python and another with more conventional programming in Python. Overall, this work is at very much initial stage and requires deeper studies to come up with a mature conclusion.

REFERENCES

[1] Janusz Jabłonowski. 2007. A case study in introductory programming. (CompSysTech '07), Boris Rachev, Angel Smrikarov, and Dimo Dimov (Eds.). ACM, New York, NY, USA, Article 82 , 7 pages

[2] Linda Grandell , Mia Peltomäki , Ralph-Johan Back , Tapio Salakoski, Why complicate things?: introducing programming in high school using Python, Proceedings of the 8th Australasian Conference on Computing Education, p.71-80, January 16-19, 2006, Hobart, Australia

[3]. Vambola Leping , Marina Lepp , Margus Niitsoo , Eno Tõnisson , Varmo Vene , Anne Villems, Python prevails, Proceedings of the International Conference on Computer Systems and Technologies and Workshop for PhD Students in Computing, June 18-19, 2009, Ruse, Bulgaria [doi>10.1145/1731740.1731833]

[4] J. Forbes and D. D. Garcia. "...but what do the top-rated schools do?": a survey of introductory computer science curricula. *SIGCSE Bull.*, 39(1):245–246, 2007.

[5] Diwaker Gupta, What is a Good First Programming Language? http://www.acm.org/crossroads/xrds10-4/firstlang.html

[6] Soloway, E. (1993), 'Should we teach students to program?', Commun. ACM 36(10), 21_24.

[7] Spohrer, J. C. & Soloway, E. (1986), 'Novice mistakes: are the folk wisdoms correct?', Commun. ACM 29(7), 624_632.

[8] McIver, L. The Effect of Programming Language on Error Rates of Novice Programmers. School of Computer Science, Monash University.

[9] Wexelblat, R. L. The Consequence of One's First Programming Language. In Proceedings of the 3rd ACM SIGSMALL Symposium and the First SIGPC Symposium on Small Systems.Palo Alto, California, 1980. pp. 52-55.

[10] R. M. Siegfried, D. Chays, and K. G. Herbert, *Will there ever be consensus on cs1?*, Proc. 2008 International Conference on Frontiers in Education: Computer Science and Computer Engineering – FECS '08, 18–23. CSREA Press, 2008.

[11] Krpan, Divna, and Ivan Bilobrk. "Introductory programming languages in higher education." *MIPRO, 2011 Proceedings of the 34th International Convention*. IEEE, 2011.

[12] McCracken, M., Almstrum, V., Diaz, D., Guzdial, M., Hagan, D., Kolikant, Y. B.-D., Laxer, C., Thomas, L., Utting, I. & Wilusz, T. (2001), 'A Multi-National, Multi-Institutional Study of Assessment of Programming Skills of First-Year CS Students', SIGCSE Bull. 33(4), 125_180.

[13] A. Pears, S. Seidman, L. Malmi, L. Mannila, E. Adams, J. Bennedsen, M. Devlin, and J. Paterson. A survey of literature on the teaching of introductory programming. In *ITiCSE-WGR '07: Working group reports on ITiCSE on Innovation and technology in computer science education*, pages 204–223, New York, NY, USA, 2007. ACM.

[14] T. Donaldson Phyton as a First Programming Language for Everyonehttp://www.cs.ubc.ca/wccce/Program03/papers/Toby.html

[15] Holly Patterson-McNeill, Experience: from C++ to Python in 3 easy steps, Journal of Computing Sciences in Colleges, v.22 n.2, p.92-96, December 2006

[16] Deveaux, D., Fleurquin, R. & Frison, P. (1999), Software engineering teaching: a "docware" approach, in 'ITiCSE '99: Proceedings of the 4th annual ITiCSE conference', Cracow, Poland, ACM Press, pp. 163_166.

[17] Lister, R., Adams, E. S., Fitzgerald, S., Fone, W., Hamer, J., Lindholm, M., McCartney, R., Moström, J. E., Sanders, K., Seppälä, O., Simon, B. & Thomas, L. (2004), A Multi-National Study of Reading and Tracing Skills in Novice Programmers, in 'ITiCSE-WGR '04: Working group reports from the ITiCSE conference', Leeds, United Kingdom, ACM Press, pp. 119_150.

[18] Anthony Robins, Janet Rountree, N. R. (2003), 'Learning and teaching programming: A review and discussion', Computer Science Education 13(2), 137_172.

[19] Milbrandt, G. (1993), 'Using problem solving to teach a programming language in computer studies', Journal of Computer Science Education 8(2), 14_19.

[20] Hagan, D. & Markham, S. (2000), Does it help to have some programming experience before beginning a computing degree program?, in 'ITiCSE '00: Proceedings of the 5th annual ITiCSE conference', Helsinki, Finland, ACM Press, pp. 25_28.

[21] A. Fuentes, J. Andersson, A. Johansson, P. Nilsson: Gender and Programming: A Case Study LTHs 3:e pedagogiska inspirationskonferens, Lund, 20050531.

[22] www.coursera.org

[23] www.udacity.com

[24] www.edx.org

[25] *www.**python**.org/~guido/*

[26] http://www.tiobe.com/index.php/content/paperinfo/tpci/Python.html

[27] http://en.wikipedia.org/wiki/Python_%28programming_language%29#Impact

View publication stats